OPEN PROTOCOL & ARCHITECTURE BLUEPRINT • ECP V0.1-MVP

git-cortex: The Epistemic Context Protocol

Git-Native Working Memory, Negative-Knowledge Tombstones, and Causal Invariant Locks for Multi-Agent Software Engineering

Specification Attribute	Normative Target
Protocol & Binary Name	`git-cortex` (CLI & Git Subcommand) / Epistemic Context Protocol (`ECP v0.1`)
Target Interop Stack	Model Context Protocol (`MCP`), Agent2Agent (`A2A`), `AGENTS.md` , Claude Code, Codex CLI, Gemini CLI, OpenCode
Storage Architecture	Tier 1: Local SQLite WAL Index (`.git/cortex/state.db`) • Tier 2: Distributed Git Notes (`refs/notes/cortex`)
AST Binding Engine	Tree-sitter Structural Symbol Hashing (`SHA-256` over normalized AST subtrees)

1. Executive Summary & Architectural Motivation

Modern agentic software engineering stacks have standardized tool execution via the Model Context Protocol (MCP), static repository instructions via `AGENTS.md` , and cross-agent task delegation via Agent2Agent (A2A). However, a critical architectural layer remains entirely unstandardized: **epistemic state and negative knowledge**. Today, when an autonomous coding agent explores a repository, tests a hypothesis, encounters a subtle failure, and reverts its edits, the causal reasoning behind that failure exists only inside an ephemeral, proprietary conversation log.

This missing primitive produces three recurring systemic failures across every commercial and open-source coding harness:

- **The Context Compaction Death Spiral:** Long-horizon debugging sessions routinely exceed context thresholds. When a harness compacts its conversation history into a lossy text summary, negative knowledge (why a specific refactor broke a downstream invariant) is pruned first. Minutes after compaction, the same agent re-implements the exact disproved approach.
- **The Cross-Harness & Sub-Agent Handoff Tax:** Switching mid-task between harnesses (e.g., from Claude Code to Gemini CLI or Codex) or spawning a specialized sub-agent forces the incoming model to start from a blank epistemic slate. Up to 70% of initial session tokens are burned re-running `grep` and `read_file` operations to reconstruct call graphs the previous agent already verified.
- **Silent Semantic Merge Conflicts:** Standard Git tracks line-level text diffs rather than causal assumptions. When Agent A merges changes relying on an implicit behavioral precondition in a shared module and Agent B alters that module days later, Git merges cleanly while introducing critical runtime regressions.

> **Core Architectural Thesis:** Source control must version *not only* what code succeeded (syntax diffs), but also *what hypotheses were proved false* (Tombstones), *what symbol relationships were verified* (Epistemic Checkpoints), and *what cross-file invariants must hold* (Causal Invariant Locks).

2. Two-Tier Storage & AST Binding Architecture

To prevent repository bloat while supporting distributed multi-agent collaboration, `git-cortex` separates high-frequency working memory from immutable repository history using a two-tier storage engine combined with deterministic Abstract Syntax Tree (AST) content addressing.

2.1 Tier 1 vs. Tier 2 Persistence Model

Raw LLM transcripts should never be committed to a Git tree. Instead, `git-cortex` maintains an ephemeral local SQLite database in `.git/cortex/state.db` during active editing and promotes compacted, high-signal epistemic records into Git's native object database under `refs/notes/cortex` upon checkpoint or commit.

Primitive	Storage Location	Lifecycle & Scope	Invalidation Trigger	Token Footprint
Working Epistemic Graph	`.git/cortex/state.db` (Local SQLite WAL)	Branch-scoped; mutated on file read, test run, or checkpoint	Branch deletion, `cortex rewind`, or AST hash drift	~400–800 tokens per handoff pack
Tombstone Node (Negative Knowledge)	Local DB + promoted to `refs/notes/cortex/tombstones`	Persists across compactions, sessions, and harnesses	Target or dependency symbol `ast_hash` changes	~80–150 tokens injected on symbol inspect
Causal Invariant Lock	`refs/notes/cortex/invariants` (Git Notes on Commit SHA)	Committed & pushed with code via `git push`	Explicit contract revision or target symbol deletion	~60–120 tokens per guarded symbol

2.2 Deterministic AST Symbol Addressing (`ast_hash`)

Binding epistemic metadata to file line numbers fails immediately because unrelated edits shift line offsets. `git-cortex` uses embedded Tree-sitter parsers to address code by canonical symbol paths and normalized AST hashes:

1. **Canonical Symbol URI:** Every function, method, struct, class, or interface is addressed as `<relative_filepath>#<scope_chain>` (for example, `src/pool/connection.rs#ConnectionPool::acquire`).
2. **Normalized AST Hash Calculation:** Before hashing a symbol's subtree, `git-cortex` strips non-semantic comments, normalizes whitespace, and serializes the CST/AST node types and identifiers into a canonical byte stream, producing a 64-bit truncated SHA-256 digest (e.g., `ast:sha256:9f84b2c10e4a11d2`).
3. **Automatic Stale-Knowledge Garbage Collection:** Every Tombstone and Invariant Lock stores the `ast_hash` of the symbols it references. When the underlying implementation of those symbols is genuinely refactored, the hash changes and `git-cortex` automatically marks stale Tombstones as `SUPERSEDED`—ensuring agents are never blocked by outdated warnings.

3. Normative Data Schemas (ECP v0.1)

All persistent objects in `git-cortex` serialize to canonical JSON (RFC 8785) for Git Notes storage and exchange over MCP and A2A protocols. The three foundational schemas required for the MVP are specified below.

3.1 Schema 1: The Tombstone Node (`TombstoneRecord`)

A `TombstoneRecord` captures a disproved hypothesis or failed code modification so no agent repeats the dead end after context compaction or harness switching.

```
{
  "$schema": "https://git-cortex.dev/schemas/v0.1/tombstone.json",
  "id": "tmb_01J9X8K4M2N6P8Q1R3S5T7V9W0",
  "schema_version": "0.1.0",
  "created_at": "2026-11-09T09:30:00Z",
  "author_harness": "claude-code/1.4.2",
  "branch_context": "fix/oauth-refresh-race",
  "base_commit_sha": "a4f82d9c10e3b72f56a190c8e7d6b5a4f3e2d1c0",
  "capture_mode": "AUTO_REVERT_DETECTED",
  "target_symbols": [
    {
      "symbol_uri": "src/auth/token_manager.ts#TokenManager.refreshSession",
      "ast_hash": "ast:sha256:4b91e08a72c5d301"
    }
  ],
  "attempted_approach": "Replaced optimistic token cache read with an exclusive
async mutex around TokenManager.refreshSession.",
  "failure_reason": "Introduced a cyclic lock acquisition when
WorkerQueue.dispatch triggers an inline token validation callback, causing
deterministic deadlock under concurrent load.",
  "verification_evidence": {
    "command": "npm test -- tests/auth/concurrent_refresh.spec.ts",
    "exit_code": 124,
    "error_signature": "ERR_TEST_TIMEOUT: WorkerQueue deadlock detected waiting on
TokenManager.mutex"
  },
  "invalidation_policy": {
    "invalidate_on_any_ast_change": [
      "src/auth/token_manager.ts#TokenManager.refreshSession",
      "src/queue/worker.ts#WorkerQueue.dispatch"
    ],
    "status": "ACTIVE"
  }
}
```

3.2 Schema 2: Cross-Harness Epistemic Handoff Envelope (`EpistemicHandoff`)

The `EpistemicHandoff` envelope compresses an agent's live working memory into a deterministic 2–4KB payload that can be ingested in milliseconds by another CLI harness or A2A sub-agent.

```
{
  "$schema": "https://git-cortex.dev/schemas/v0.1/handoff.json",
  "handoff_id": "hnd_01J9X9A1B2C3D4E5F6G7H8J9K0",
  "schema_version": "0.1.0",
  "head_commit_sha": "a4f82d9c10e3b72f56a190c8e7d6b5a4f3e2d1c0",
  "working_tree_dirty_hash": "wt:sha256:81c2f3a900d4e112",
  "objective": "Eliminate race condition in OAuth token refresh without blocking
WorkerQueue dispatchers.",
  "verified_call_graph": [
    {
      "caller": "src/queue/worker.ts#WorkerQueue.dispatch",
      "callee": "src/auth/token_manager.ts#TokenManager.getValidToken",
      "relationship": "SYNCHRONOUS_PRECHECK",
      "verified_ast_hash": "ast:sha256:7d12e9f04a8c33b1"
    }
  ],
  "active_hypotheses": [
    {
      "hypothesis_id": "hyp_02",
      "statement": "Use singleflight promise deduplication keyed by tenant_id
inside TokenManager.refreshSession instead of a mutex lock.",
      "confidence": 0.88,
      "status": "IN_PROGRESS",
      "next_verification_step": "Run tests/auth/concurrent_refresh.spec.ts with 50
parallel workers."
    }
  ],
  "active_tombstone_ids": [
    "tmb_01J9X8K4M2N6P8Q1R3S5T7V9W0"
  ],
  "do_not_re_read_unchanged": [
    "src/config/env.ts",
    "src/logging/telemetry.ts"
  ]
}
```

3.3 Schema 3: Causal Invariant Lock (`InvariantContract`)

An `InvariantContract` is attached to a Git commit via `refs/notes/cortex/invariants` and specifies architectural constraints that downstream agents and PRs must respect.

```
{
  "$schema": "https://git-cortex.dev/schemas/v0.1/invariant.json",
  "invariant_id": "inv_01J9X9Z9Y8X7W6V5U4T3S2R1Q0",
  "schema_version": "0.1.0",
  "commit_sha": "f3e2d1c0a4f82d9c10e3b72f56a190c8e7d6b5a4",
  "guarded_symbol": {
    "symbol_uri": "src/billing/ledger.ts#LedgerService.issueRefund",
    "ast_hash": "ast:sha256:3c90a12b5e7f8804"
  },
  "severity": "BLOCKING",
  "assertion": "LedgerService.issueRefund assumes idempotency_key has already been
atomically reserved in Redis by IdempotencyMiddleware. Calling issueRefund
directly from cron workers without acquiring a reservation causes double-
refunds.",
  "upstream_dependencies": [
    {
      "symbol_uri": "src/gateway/idempotency.ts#IdempotencyMiddleware.reserveKey",
      "expected_ast_hash": "ast:sha256:8f9a11c40d2e7765"
    }
  ],
  "verification_test_cmd": "npm test -- tests/billing/idempotent_refund.spec.ts"
}
```

4. Model Context Protocol (MCP) Server Tool Definitions

To achieve immediate Day-1 adoption across Claude Code, Codex CLI, Gemini CLI, Cursor, and OpenCode without waiting for upstream PRs, `git-cortex` runs as a local stdio MCP server (`git-cortex mcp-serve`). It exposes five deterministic tools with strict token budgets.

MCP Tool Name	Invocation Trigger	Input Parameters	Output Payload
`cortex_inspect_symbol`	Called before modifying any function or file	`filepath` (string), `symbol_name` (optional string)	Active `TombstoneRecord[]` and `InvariantContract[]` bound to the file/symbol
`cortex_record_tombstone`	Called when an experiment fails or a diff is reverted	`target_symbols[]` , `attempted_approach` , `failure_reason` , `test_cmd`	Created `tombstone_id` and bound `ast_hash` digest
`cortex_checkpoint_state`	Called after verifying call graphs or before context compaction	`objective` , `verified_edges[]` , `active_hypotheses[]`	Updated `handoff_id` and working graph stats

MCP Tool Name	Invocation Trigger	Input Parameters	Output Payload
`cortex_handoff_pack`	Called at session start or prior to switching models/sub-agents	`mode` (`"IMPORT" | "EXPORT"`), `branch` (optional)	Compact `EpistemicHandoff` JSON (≤ 4 KB)
`cortex_verify_invariants`	Called before committing or creating a pull request	`diff_scope` (`"STAGED" | "WORKING_TREE" | "BRANCH"`)	List of violated or drifted `InvariantContract` locks and required tests

4.1 Normative MCP JSON-RPC Tool Declarations

Below is the normative MCP `tools/list` schema definition for the two highest-frequency tools: `cortex_inspect_symbol` and `cortex_record_tombstone`.

```
{
  "tools": [
    {
      "name": "cortex_inspect_symbol",
      "description": "Retrieve active negative-knowledge Tombstones (failed past
approaches) and Causal Invariant Locks for a file or AST symbol BEFORE planning or
applying edits.",
      "inputSchema": {
        "type": "object",
        "properties": {
          "filepath": {
            "type": "string",
            "description": "Repository-relative path to the source file (e.g.,
'src/auth/token_manager.ts')."
          },
          "symbol_name": {
            "type": "string",
            "description": "Optional canonical class/function identifier (e.g.,
'TokenManager.refreshSession')."
          }
        },
        "required": ["filepath"]
      }
    },
    {
      "name": "cortex_record_tombstone",
      "description": "Record a disproved hypothesis or failed code modification as
an AST-bound Tombstone so neither this agent (post-compaction) nor downstream
agents repeat the dead end.",
      "inputSchema": {
        "type": "object",
        "properties": {
          "target_symbols": {
            "type": "array",
            "items": { "type": "string" },
            "description": "List of symbol URIs modified during the failed
attempt."
          },
          "attempted_approach": {
            "type": "string",
            "description": "Concise technical summary of the change that was
attempted."
          },
          "failure_reason": {
            "type": "string",
            "description": "Root cause explaining why the attempt failed (test
failure, deadlock, type violation)."
          },
          "test_command": {
            "type": "string",
            "description": "Command that demonstrated the failure."
          },
          "error_signature": {
            "type": "string",
            "description": "Key assertion or error output line."
```

5. Passive Capture Engine & Git Lifecycle Integration

Relying exclusively on an LLM to voluntarily log its own failures is brittle; when an agent is struggling with a bug, it may forget to call `cortex_record_tombstone` before reverting a file. Therefore, `git-cortex` includes a deterministic **Passive Capture Engine** that observes the filesystem and test runner directly.

5.1 The Automatic "Edit → Fail → Revert" Detector

When initialized in a repository (`git cortex init`), a lightweight background daemon or test wrapper (`git cortex exec -- <test-cmd>`) monitors working tree mutations:

1. **Dirty Diff Snapshot:** When the working tree diverges from `HEAD`, the daemon parses modified files with Tree-sitter and records a transient candidate snapshot in `.git/cortex/state.db` mapping touched symbols (`symbol_uri` + pre/post `ast_hash`).
2. **Failure Correlation:** If a build or test command exits with a non-zero code while those symbols are dirty, `git-cortex` attaches the exit code and stderr tail to the candidate snapshot.
3. **Revert Promotion:** If the agent subsequently restores those symbols back to their original `ast_hash` (via `git checkout`, `git restore`, or an inverse file edit), `git-cortex` automatically promotes the candidate into an active `AUTO_REVERT_DETECTED` Tombstone without requiring a single LLM tool call.

5.2 Split-Brain Safe Rollback (`git cortex rewind`)

When an agent goes down a rabbit hole for 20 minutes, running a raw `git reset --hard` resets the files on disk but leaves the agent's context window polluted—or wipes out the lesson of why that 20-minute branch failed. Running `git cortex rewind <checkpoint-id>` executes an atomic four-step recovery:

- Synthesizes a summary `TombstoneRecord` covering the diff between `<checkpoint-id>` and the aborted working state.
- Restores the Git working tree and cleans untracked build artifacts back to `<checkpoint-id>`.
- Rolls back `.git/cortex/state.db` to the target checkpoint's epistemic graph.
- Appends the newly minted Tombstone to the restored checkpoint so the resumed agent starts with a clean working tree *plus* explicit immunity against the rolled-back path.

5.3 Native Git Hooks & Note Synchronization

- **`pre-commit` Hook:** Runs `git-cortex verify-invariants --staged` in under 35ms. If a staged diff modifies an upstream dependency of a `BLOCKING` `InvariantContract`, the hook halts the commit and prints the contract assertion and verification test command.
- **`post-commit` Hook:** Migrates verified session Tombstones and new `InvariantContract` records from `.git/cortex/state.db` into `refs/notes/cortex` bound to the newly created commit SHA.

- **`post-merge` / `post-checkout` Hook:** Re-indexes incoming Git Notes from `refs/notes/cortex` into the local SQLite cache and prunes any Tombstones whose target `ast_hash` changed in the merged commits.

6. Cross-Protocol Integration (`AGENTS.md` & Google `A2A`)

`git-cortex` is designed as a complementary glue layer across existing Linux Foundation / AAIF standards rather than a competing framework.

6.1 Standard `AGENTS.md` Bootstrap Block

Running `npx git-cortex init` automatically appends the following normative instruction block to the repository's `AGENTS.md` (and `CLAUDE.md`), ensuring every compliant harness uses the protocol immediately:

```
## Epistemic Memory & Invariant Protocol (git-cortex)
- **Session Start:** Call `cortex_handoff_pack(mode="IMPORT")` before running
broad repo searches to load verified call-graphs and active hypotheses.
- **Before Editing Symbols:** Call `cortex_inspect_symbol(filepath)` before
modifying existing functions. Respect all ACTIVE Tombstones and Invariant
Contracts.
- **On Failed Experiments:** Whenever a test fails and you revert or abandon an
approach, call `cortex_record_tombstone` before trying an alternative.
- **Before Committing:** Call `cortex_verify_invariants(diff_scope="STAGED")` to
verify cross-file causal contracts.
```

6.2 Google A2A (Agent2Agent) Task Envelope Extension

When a coordinator agent delegates a coding sub-task to a remote or local specialist agent over the A2A protocol, it embeds the compact `EpistemicHandoff` object directly inside the A2A Task metadata extension field:

```
{
  "jsonrpc": "2.0",
  "method": "tasks/send",
  "params": {
    "id": "task_auth_refactor_09",
    "message": {
      "role": "user",
      "parts": [{ "type": "text", "text": "Implement singleflight token refresh in
TokenManager." }]
    },
    "metadata": {
      "https://git-cortex.dev/a2a-extension/v0.1": {
        "handoff_id": "hnd_01J9X9A1B2C3D4E5F6G7H8J9K0",
        "epistemic_handoff": { "$ref": "#/components/schemas/EpistemicHandoff" }
      }
    }
  }
}
```

7. MVP Implementation Architecture & 30-Day Launch Roadmap

To maximize performance, portability, and single-binary distribution (`npx git-cortex`, Homebrew, Cargo), the reference MVP is architected as a modular Rust workspace with zero runtime dependencies.

Rust Crate / Module	Key Dependencies	Core Responsibility
`cortex-ast`	`tree-sitter`, `sha2` (TS, Rust, Python, Go grammars)	Parses source files into CSTs, extracts canonical `symbol_uri` paths, and computes comment-insensitive normalized `ast_hash` digests.
`cortex-storage`	`rusqlite` (bundled), `git2` (libgit2)	Manages `.git/cortex/state.db` WAL queries and reads/writes canonical JSON blobs to `refs/notes/cortex` without shelling out to slow processes.
`cortex-watcher`	`notify`, `similar`	Passive capture daemon and `cortex exec` test runner wrapper that detects the *Edit → Fail → Revert* cycle automatically.
`cortex-mcp`	`tokio`, `serde_json`	Zero-latency stdio JSON-RPC 2.0 MCP server exposing the 5 normative `cortex_*` tools to any coding harness.
`cortex-cli`	`clap`	User-facing CLI providing `init`, `status`, `handoff`, `tombstones`, `rewind`, and `verify-invariants` commands.

7.1 Phased 4-Week Execution Plan

- **Week 1 (Core Engine & Tree-Sitter Hashing):** Implement `cortex-ast` with TypeScript, Python, Rust, and Go grammar support and `cortex-storage` backed by SQLite + `refs/notes/cortex`. Verify sub-15ms AST hashing on 10,000-line files.
- **Week 2 (MCP Server & Passive Revert Capture):** Build `cortex-mcp` implementing all five MCP tools and `git cortex exec` to auto-capture Tombstones on test failure + revert. Validate end-to-end with Claude Code and Gemini CLI.
- **Week 3 (Cross-Harness Handoff & Invariant Locks):** Implement `git cortex handoff`, `git cortex rewind`, and the `pre-commit` semantic invariant verifier. Benchmark token savings across a 20-task multi-harness debugging benchmark.
- **Week 4 (Viral Open-Source Launch & ECP v0.1 RFC):** Publish single-command installers (`npx git-cortex init` / `brew install git-cortex`), release the split-screen Claude-to-Gemini/Codex handoff demo, and open the `ECP v0.1` specification repository for upstream harness integrations.